Jaypee Institute of Information Technology, Noida

UNIX

Project-Based Learning Report: System Manager

Kumar Aryan 2401030217

Naina Garg 2401030207

Introduction

- **System Manager:** Real-time Linux System Monitoring Dashboard

- ## Project Goal

  To develop a lightweight, modular, real-time system monitoring dashboard using only shell scripting and the Kitty terminal emulator, enabling Linux users to view CPU, RAM, GPU, temperature, process list, and other system statistics using a single command.

- ## Motivation

  Traditional GUI system monitors consume more resources than they monitor. Linux power users, especially those on tiling window managers, require:

  - Minimal overhead
  - Command-line accessibility
  - Modular design
  - Real-time updates

  This project solves these problems using pure UNIX shell scripting.

Project Overview and Problem Statement

• Problem Description

Modern operating systems often come with heavy, graphical resource monitors that consume significant resources themselves. For Linux users, especially those focusing on performance or using tiling window managers, there is a need for a **minimalist, low-overhead, and highly customizable** system monitoring solution that can display key metrics in real-time widgets. Relying solely on running commands like `top` or `free` manually is inefficient and requires constant context switching.

• Proposed Solution

The **System Manager** project addresses this by developing a suite of interconnected shell scripts (`.sh`) designed to:

1. Read system data using standard Linux utilities (`top`, `free`, `ps`, `df`, `sensors`, `nvidia-smi`, `lscpu`).
2. Display the output in real-time, refreshing within individual terminal windows.
3. Utilize the **Kitty terminal emulator's** advanced features (window classification, title setting, transparency) to launch these scripts as modular, focused dashboard "widgets."

Implementation and Technical Design

• Modular Architecture

The solution employs a modular design, where each system component (CPU, RAM, GPU, etc.) is monitored by its own dedicated script and window. This allows for easy maintenance and scaling.

Each system component has a dedicated script:

- `cpu.sh` — CPU usage, load average
- `ram.sh` — memory usage
- `gpu.sh` — GPU usage (Nvidia/AMD/Intel auto-detection)
- `disk.sh` — disk & filesystem health
- `tasks.sh` — active processes
- `clock.sh` — time & uptime
- `systemmanager.sh` — master launcher script

This makes the system scalable and easy to maintain.

• Orchestration (`systemmanager.sh`)

The master script utilizes Kitty's parameters (`-class`, `--title`, `--override`) to enforce specific aesthetic and layout rules for each widget. The use of '`&`' runs each monitor in the background, allowing the master script to launch all widgets concurrently.

• Hardware (GPU Example)

The `gpu.sh` script demonstrates a crucial design decision: **hardware abstraction**. It detects the presence of `nvidia-smi` (for Nvidia) or checks `lspci` and `/sys/class/drm` for AMD/Intel, tailoring the subsequent data retrieval command to the specific hardware.

```
# Snippet from gpu.sh demonstrating abstraction
case $GPU in
    Nvidia)
```

```
        nvidia-smi --query-gpu=name,utilization.gpu,...
      ;;
    AMD)
        sudo cat /sys/class/drm/card0/device/gpu_busy_percent
      ;;
    # ...
esac
```

Evaluation and Conclusion

- ## Results

  The System Manager successfully functions as a **minimalist, live system dashboard**. It meets the objectives of providing real-time resource usage (CPU, RAM, Processes), system health checks (Disk, Uptime, Temp), and a clock, all without consuming excessive system resources. The use of shell scripts ensures maximum performance efficiency.

- ## Future Work

  1. **Configuration File:** Implement a simple `.conf` file to allow users to easily configure refresh rates, widget sizes, or opacity without editing the `.sh` files.
  2. **Network Monitoring:** Add a new script to display real-time network usage (e.g., bandwidth speed and active connections).
  3. **Process Control:** Introduce interactive elements to allow the user to easily kill processes listed in `tasks.sh` directly from the widget (this would require careful handling of input within the infinite loop).
  4. Hyprland: Using Hyprland to enhance the looks of each widget and change how it interacts with the adjacent widget, also fixing its spawn location.

  This project provided valuable experience in shell scripting, system utility command line usage, and the implementation of modular software design.

References

- **Stack Overflow: www.stackoverflow.com**
- **Kitty Terminal Emulator Documentation: https://sw.kovidgoyal.net/kitty**
- **Linux Manual Pages: man top, man free, man df, man ps, man watch, man sensors etc.**
- **Geeks for Geeks: https://www.geeksforgeeks.org/category/linux-unix**